AICTE VERIFIED STUDENT REGISTER

Vibecoding Prompt Pack and Build Order

Copy-ready prompts to run a disciplined build with an AI coding agent, from plan to final QA.

Document: 09-PROMPTS Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Operating model

Use one coding agent as the primary editor. Give it one milestone, its source PDFs and acceptance criteria. Do not ask it to implement the whole 33-page system in one request. After each milestone, inspect diffs, run the app, execute tests and create a Git checkpoint.

2. Prompt 0 - read and plan only

```
Read these source attachments before coding:
- 01_Problem_Statement_and_Product_Scope.pdf
- 02_Tech_Stack_and_Repository_Spec.pdf
- 05_System_Architecture_and_Workflows.pdf

Do not modify or create application files yet. Return: requirements summary,
architecture, role/permission map, collection model, API contracts, milestone
plan, test plan, risks and an explicit TBD list. Mark each statement as source
requirement, proposed implementation or unresolved. Do not invent official seat
quotas, real API access, credentials, legal requirements or measured performance.
Wait for review before coding.
```

3. Prompt 1 - foundation

```
Implement only Phase 1 from the approved plan. Use the exact stack in
02_Tech_Stack_and_Repository_Spec.pdf. Inspect the existing repository first.
Create React/JavaScript/Bootstrap client, Express server, environment example,
MongoDB connection, health check, error handler, README and basic tests.
Do not implement admission business logic yet. Run checks and report the exact
commands/results, changed files, new packages, assumptions and remaining issues.
```

4. Prompt 2 - authentication

```
Attach 03_Authentication_and_Authorization_Spec.pdf and 04_Role_Features_and_Button_Inventory.pdf.
Implement login/logout, server-side sessions, password hashing, current-user
endpoint and backend RBAC/scope checks. Use only seeded synthetic accounts.
Do not proceed to CSV uploads. Test invalid credentials, 401, 403, cross-college
access, regional scope, disabled account and logout invalidation. Report failures honestly.
```

5. Prompt 3 - upload and validation

```
Attach 06_Database_and_API_Contract.pdf, 07_Validation_Seat_Ledger_and_Conflicts.pdf,
and 08_Security_Privacy_Testing_and_Acceptance.pdf. Implement CSV ingestion,
batch/staging records, row-level issues, correction and revalidation only. Keep
all staged data out of the official register. Use mock external adapters. Test bad
file, missing fields, invalid demo ID format, duplicate rows and upload limits.
```

6. Prompt 4 - seat ledger and certification

```
Implement only the seat ledger and maker-checker commit in document 07. Use
MongoDB transactions where required and document/configure a replica set or
supported hosted topology. Demo capacities must come from synthetic config, not
real policy defaults. The commit must recheck blockers and policy version; update
seat counter, enrolment and audit/history consistently; rollback on failure. Test
capacity overflow, concurrent last-seat attempts and forced transaction failure.
```

7. Prompt 5 - conflict and dashboards

```
Attach 04, 05, 06, 07 and 08. Implement cross-college conflict review and the
role-specific dashboards using real data from the local database. Preserve role
scopes on the server. Do not use hard-coded dashboard metrics. A cross-college
match is a case for review, not a fraud finding. Test transfer, unresolved case,
role violations and region/institution data isolation.
```

8. Prompt 6 - risk, consent, QR and audit

```
Implement one capability at a time using documents 04, 06, 07 and 08. First
rule-based risk flags with reasons; second consent approval/denial/revocation;
third mock QR status; fourth hash-chain audit integrity. Never call the rule-based
score machine learning, an unsigned QR a signed credential, or a mock adapter a
real government integration. Add tests for false positives, expiry, revocation,
malformed QR references and modified audit entries.
```

9. Prompt 7 - independent QA

```
Attach 08_Security_Privacy_Testing_and_Acceptance.pdf. Do not add features.
Inspect code, run tests and report pass/fail/not run with exact commands. Check
role authorization on the server, data scope, transaction safety, upload limits,
secret handling, consent expiry/revocation, QR boundaries and audit integrity.
Fix confirmed defects only, rerun affected tests, and provide residual risks. Do
not claim production readiness or report unexecuted tests as passed.
```

10. Debugging prompt

```
Observed behaviour:
[What happened]
Expected behaviour:
[What should happen]
Exact command and error:
[Paste output]

Inspect the existing code and identify the root cause before editing. Explain the
responsible file/function, smallest safe fix and likely regression risks. Apply only
the minimal correction, run relevant tests and report the real result. Do not
rewrite unrelated files, change the stack or suppress the error.
```

11. Review prompt after any change

```
Review the diff for this milestone against its PDF specification. Identify
requirements not met, dead buttons, APIs with no UI or UI with no API, missing
authorization, inconsistent fields, unhandled errors, untested code and scope drift.
Do not add new features. Present findings by severity and cite the specific file,
route or function. Fix only confirmed defects after I approve the patch.
```

12. Per-milestone report the agent must return

Report item	Required detail
Summary	What is implemented and what remains unimplemented.
Changed files	Path and purpose for each material change.
Dependencies	New package, reason, environment/config changes.
Commands	Exact tests/build/start commands executed.
Test results	Pass/fail/not run; include failure summaries.
Assumptions/TBD	Anything not supported by source or not decided.
Security / privacy	Relevant protections and what is not yet protected.
Next step	The single next milestone; no unapproved scope expansion.

13. Git checkpoint discipline

1. Before each milestone, run git status and create a commit/branch for the known-good state.
2. Review the diff after the agent edits; reject unexplained wholesale rewrites.
3. Run the milestone tests yourself or inspect the actual captured output.
4. Commit only after acceptance criteria pass.
5. If a change breaks a known-good milestone, revert or fix the narrow diff before moving forward.

14. Final anti-hallucination check

- Ask the agent to name the PDF and section supporting each product requirement.
- Require it to label its own engineering choices as proposed when they are not in source.
- Ask for proof of live integration before accepting any claim about external APIs.
- Ask for measured test evidence before accepting performance or accuracy numbers.
- Verify that seat rules have a source reference/version or are labelled synthetic configuration.
- Verify that all role buttons are listed, wired, permission-checked and tested.